**Taller Práctico de Repaso y Consolidación en C++**

Alexis Maier
E.E.S.T. N° 1 “Eduardo Ader” — Vicente López
Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B
Profesor: Mansilla Muñoz York Elías
Fecha: 01 de octubre de 2026

## Introducción

La presente actividad consolida de forma práctica, analítica y sin fricción teórica los tres pilares fundamentales de bajo nivel trabajados durante la primera mitad del ciclo lectivo: recursividad, búsqueda secuencial e intercambio físico con punteros. El objetivo es comprender cómo el microprocesador y la memoria RAM interactúan con los algoritmos implementados, aplicando las bases de software necesarias para procesar datos de sensores y módulos de hardware en tiempo real dentro del Proyecto Integrador Anual (Expotécnica 2026).

## Desarrollo

**RETO 1 — Recursividad.** Se implementa la función *sumaRecursiva(int n)* que se invoca a sí misma sumando *n + sumaRecursiva(n-1)* hasta el caso base *n <= 0*. La salida obtenida es: Suma acumulada de 1 hasta 5: 15.

**RETO 2 — Búsqueda secuencial.** Se recorre celda por celda un arreglo contiguo de 10 enteros buscando el valor 18. Al encontrarlo se imprime el índice y su dirección física en RAM hexadecimal, interrumpiendo el ciclo con *break* para optimizar el tiempo de ejecución.

**RETO 3 — Intercambio con punteros.** Se intercambian dos variables enteras pasando sus direcciones físicas (&) a un puntero y desreferenciándolo (*). La salida confirma el intercambio físico de las celdas de memoria.

## Cuestionario Técnico de Control

1. **Caso Base:** Si una función recursiva carece de caso base de corte, la pila de llamadas (Stack) se desborda produciendo un *Stack Overflow*, colapsando la memoria RAM.

2. **Eficiencia:** La instrucción *break* optimiza el rendimiento del microprocesador al interrumpir el ciclo O(n) en cuanto se encuentra el elemento, evitando recorrer el resto del arreglo.

3. **Direccionamiento:** Mediante el operador & se observa que las direcciones de elementos consecutivos del arreglo se incrementan en 4 bytes (tamaño de un *int*), comprobando que ocupan celdas contiguas.

4. **Variables Temporales:** La variable temporal guarda el primer valor mientras se copia el segundo al primero, evitando la pérdida del dato original durante el intercambio.

5. **Autoevaluación:** El reto más complejo fue el intercambio con punteros, resuelto comprendiendo el pasaje por dirección (&) y la desreferenciación (*), modificando directamente las celdas originales.

## Conclusión

La actividad demuestra que el control eficiente de la memoria de bajo nivel y el registro riguroso de versiones son pilares del software integrador. Las bases adquiridas permiten procesar datos de sensores y módulos de hardware en tiempo real para el prototipo del Proyecto Integrador Anual.

## Referencias

cplusplus.com. (s.f.). Recursion in C++. https://cplusplus.com/doc/tutorial/functions2/

Microsoft Learn. (s.f.). Arrays y punteros en C++. https://learn.microsoft.com/es-es/cpp/cpp